Трансляция презентации (во время очных лекций)

https://clck.ru/3D3Efj

При просмотре презентации в PDF для отображения анимаций на слайдах необходимо использовать Acrobat Reader, KDE Okular, PDF-XChange, Foxit Reader, браузер Firefox. Для браузеров на движке Chrome (Edge, Яндекс, Opera, ...) необходимо использовать PDF.js с опцией «Enable active content (JavaScript) in PDFs».

Промышленное программирование
Тема 2
Разработка модульных программ на C++

Гаврилов Андрей Геннадьевич

кандидат технических наук, доцент

Кафедра Информационных технологий и вычислительных систем
МГТУ «СТАНКИН»

Обновлено: 12 марта 2026 г.

План лекции

1 Разделение кода на .h и .cpp файлы

2 Статические библиотеки

3 Динамические библиотеки

Напишем математическую программу

main.cpp

```
//include iostream cmath tuple exception clocale print

std::tuple<double,double> quadratic_equation(double a, double b, double c)
{
  double D = b*b-4*a*c;
  if (D>0)
  return std::make_tuple(  (-b + sqrt(D))/2/a , ( -b - sqrt(D))/2/a  );
  else
  throw std::exception("D < 0 error!");
}

int main()
{
  setlocale(LC_ALL, "Russian");
  int a,b,c;
  std::cin>>a>>b>>c;
  auto [X1,X2] = quadratic_equation(a,b,c);
  std::println("x1={} x2={}",X1, X2);
  return 0;
}
```

Соберём и запустим её

CMakeLists.txt

```
cmake_minimum_required(VERSION 4.2)
project ("quadratic equation")
set (CMAKE_CXX_STANDARD 23) #Нам нужна современная версия языка для std::println
add_executable(qe main.cpp)
```

```
PS ...> cmake -B build
-- Build files have been written to: .../build
PS ...> cmake --build build
qe.vcxproj - > .../build/Debug/qe.exe
1 -5 6   ⟸ Вводим коэф. уравнения
x1=3 x2=2     ⟸ Все работает!
```

Раздел 1

Разделение кода на .h и .cpp файлы

Основные форматы файлов

Заголовочные файлы (с расширением .h, .hpp, .hxx) —

это файлы, которые содержат объявления (декларации) функций, классов, переменных, констант и других сущностей, но не содержат их реализацию.

Файлы кода (с расширением .cpp, .cc, .cxx) —

это файлы, которые содержат реализацию (определения) функций, методов классов и других сущностей, объявленных в заголовочных файлах.

func.h

```
void foo();//прототип

class Stankin //описание класса
{             //и его функций
  int students_count = 0;
  public:
  void bar();
}
```

func.cpp

```
#include<func.h>
void foo() //реализация foo()
{
  std::println("foooooo");
}
void Stankin::bar() //реализация
{                   //Stankin::bar()
  std::println("baaarrr");
}
```

Основные форматы файлов

Заголовочные файлы (с расширением .h, .hpp, .hxx) —

это файлы, которые содержат объявления (декларации) функций, классов, переменных, констант и других сущностей, но не содержат их реализацию.

Файлы кода (с расширением .cpp, .cc, .cxx) —

это файлы, которые содержат реализацию (определения) функций, методов классов и других сущностей, объявленных в заголовочных файлах.

func.h

```
void foo();//прототип

class Stankin //описание класса
{             //и его функций
  int students_count = 0;
  public:
  void bar();
}
```

func.cpp

```
#include<func.h>
void foo() //реализация foo()
{
  std::println("foooooo");
}
void Stankin::bar() //реализация
{                   //Stankin::bar()
  std::println("baaarrr");
}
```

Сделаем новую библиотеку для исходной программы

mymath.h

```
#include <tuple>
std::tuple<double,double> quadratic_equation(double, double, double);
```

mymath.cpp

```
#include <cmath>
#include <exception>
#include "mymath.h"

std::tuple<double,double> quadratic_equation(double a, double b, double c)
{
  double D = b*b-4*a*c;
  if (D>0)
  return std::make_tuple(  (-b + sqrt(D))/2/a , ( -b - sqrt(D))/2/a  );
  else
  throw std::exception("D < 0 error!");
}
```

Немного подправим main.cpp и CMakeLists.txt

main.cpp

```
//include <print> <iostream> <clocale>
#include "mymath.h" // <-Наша библиотека
//cmath tuple exception   больше не нужны тут

int main()
{
  setlocale(LC_ALL, "Russian");
  int a,b,c;
  std::cin>>a>>b>>c;
  auto [X1,X2] = quadratic_equation(a,b,c);
  std::println("x1={} x2={}",X1, X2);

  return 0;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 4.2)
project ("quadratic equation")
set (CMAKE_CXX_STANDARD 23)
add_executable(qe main.cpp mymath.cpp) #Добавляем сюда все cpp
```

Проверяем результат

```
PS ...> cmake -B build
 ...
-- Build files have been written to: .../build

PS ...> cmake --build build/
 ...
qe.vcxproj -> .../build/Debug/qe.exe

PS ...> ./build/Debug/qe.exe
/colororange1 -5 6
x1=3 x2=2   <== все работает!


PS ...> tree ./build/qe.dir/Debug /F
C:/.../BUILD/QE.DIR/DEBUG
|   main.obj                <== наши obj
|   mymath.obj              <== наши obj
|    ...
 ...
```

Проверяем результат

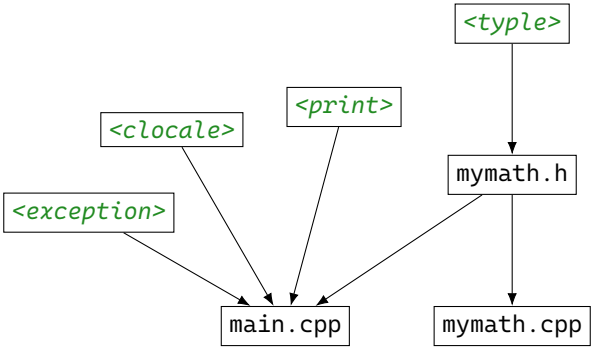
```
PS ...> cmake -B build
 ...
-- Build files have been written to: .../build

PS ...> cmake --build build/
 ...
qe.vcxproj -> .../build/Debug/qe.exe

PS ...>  ./build/Debug/qe.exe
/colororange1 -5 6
x1=3 x2=2  ⟸ все работает!
```

```
PS ...>   tree ./build/qe.dir/Debug /F
C:/.../BUILD/QE.DIR/DEBUG
|   main.obj                    ⟸ наши obj
|   mymath.obj                  ⟸ наши obj
|    ...
 ...
```

Проверяем результат

```
PS ...> cmake -B build
 ...
-- Build files have been written to: .../build

PS ...> cmake --build build/
 ...
qe.vcxproj -> .../build/Debug/qe.exe

PS ...>  ./build/Debug/qe.exe
/colororange1 -5 6
x1=3 x2=2  ⟸ все работает!


PS ...>   tree ./build/qe.dir/Debug /F
C:/.../BUILD/QE.DIR/DEBUG
|   main.obj                    ⟸ наши obj
|   mymath.obj                ⟸ наши obj
|    ...
 ...
```

Схема включений



Усложним проект

mymath.h

```
#include <tuple>
std::tuple<double,double> quadratic_equation(double, double, double);

bool isEven(int n)
{
  return n % 2 == 0;
}
```

main.cpp

```
...
int main()
{
  ...

  std::println( "x1 is {}", isEven(X1)?"even":"odd" );

  return 0;
}
```

Собираем

```
PS ..> cmake --build build/
Версия MSBuild 18.3.0-release-26070-10+3972042b7 для .NET Framework

Проверка источников на наличие зависимостей модуля…
mymath.cpp
main.cpp
Компиляция…
main.cpp
mymath.cpp
 ...
mymath.obj : error LNK2005: "bool __cdecl isEven(int)" (?isEven@@YA_NH@Z)
уже определен в main.obj
fatal error LNK1169: обнаружен многократно определенный
символ - один или более
```

Решение проблемы — inline

mymath.h

```
#include <tuple>
std::tuple<double,double> quadratic_equation(double, double, double);

inline  bool isEven(int n)
{
  return n % 2 == 0;
}
```

```
PS ...> cmake --build build/
 ...
qe.vcxproj -> C:.../build/Debug/qe.exe

PS ...> ./build/Debug/qe.exe
/colororange1 -9 20
x1=5 x2=4
x1 is not odd ⟸
```

Встроенные функции (inline)

Inline (подстановка) —

это указание компилятору, что код функции нужно вставить непосредственно в место вызова, вместо генерации инструкции вызова (call).

Основные цели

- Оптимизация скорости — убираются накладные расходы на вызов функции (создание стекового кадра, передача аргументов, возврат)
- Использование в заголовочных файлах — позволяет определять функции в .h файлах без ошибок множественного определения

```
// Без inline
int add(int a, int b) {
  return a + b;
}

int main() {
  int x = add(5, 3);
  // Здесь будет инструкция CALL
  return 0;
}
```

```
// С inline
inline int add(int a, int b) {
  return a + b;
}

int main() {
  int x = add(5, 3);
  // Компилятор заменит на: int x = 5 + 3;
  return 0;
}
```

Добавим константу в библиотеку

mymath.h

```
#include <tuple>
std::tuple<double,double> quadratic_equation(double, double, double);

inline bool isEven(int n) {
  return n % 2 == 0;
}

//не будет работать!!
double half_PI = 1.5707963268;
```

Варианты:

```
//1. будет
const double half_PI = 1.5707963268;
//2. будет c++11
constexpr double half_PI = 1.5707963268;
//3. будет даже так!  c++17
inline double half_PI = 1.5707963268;
//4. будет, но не стоит
static double half_PI = 1.5707963268;
```

Добавим константу в библиотеку

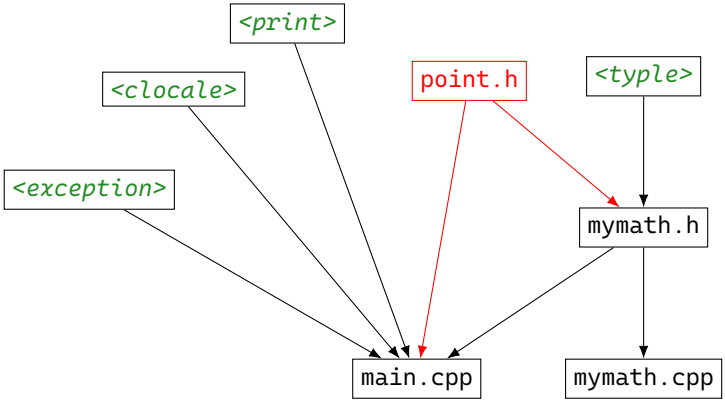
mymath.h

```
#include <tuple>
std::tuple<double,double> quadratic_equation(double, double, double);

inline bool isEven(int n) {
  return n % 2 == 0;
}

//не будет работать!!
double half_PI = 1.5707963268;
```

Варианты:

```
//1. будет
const double half_PI = 1.5707963268;
//2. будет c++11
constexpr double half_PI = 1.5707963268;
//3. будет даже так!  c++17
inline double half_PI = 1.5707963268;
//4. будет, но не стоит
static double half_PI = 1.5707963268;
```

Добавим новую библиотеку



Point.h

```
1 struct Point
2 {
3   double x;
4   double y;
5 };
```

Собираем с новой библиотекой

```
PS ...> cmake --build build/
Версия MSBuild 18.3.0-release-26070-10+3972042b7 для .NET Framework

Проверка источников на наличие зависимостей модуля…
mymath.cpp
main.cpp
Компиляция…
mymath.cpp
main.cpp
C:/.../point.h(1,8):
error C2011: Point: переопределение типа "struct"
[C:.../build/qe.vcxproj]
(компиляция исходного файла "../main.cpp")
C:/.../point.h(1,8):
см. объявление "Point"
```

Стражи заголовков

Стражи заголовков (Include Guards) —

конструкция препроцессора, защищающая от повторного включения одного и того же заголовочного файла. Предотвращают множественные определения функций и классов, которые вызывают ошибки компиляции.

Два способа реализации:

Через директиву `#pragma once`

Point.h

```
#pragma once //просто,
//но не стандартно
struct Point
{
  double x;
  double y;
};
```

Через конструкцию `#ifndef ... #endif`

Point.h

```
#ifndef POINT_H //сложнее, но всегда работает
#define POINT_H //нужно придумать такое имя,
//которое никогда не повторится
struct Point
{
  double x;
  double y;
};

#endif //POINT_H
```

Стражи заголовков

Стражи заголовков (Include Guards) —

конструкция препроцессора, защищающая от повторного включения одного и того же заголовочного файла. Предотвращают множественные определения функций и классов, которые вызывают ошибки компиляции.

Два способа реализации:

Через директиву `#pragma once`

Point.h

```
#pragma once //просто,
//но не стандартно
struct Point
{
  double x;
  double y;
};
```

Через конструкцию `#ifndef ... #endif`

Point.h

```
#ifndef POINT_H //сложнее, но всегда работает
#define POINT_H //нужно придумать такое имя,
//которое никогда не повторится
struct Point
{
  double x;
  double y;
};

#endif //POINT_H
```

Стражи заголовков

Стражи заголовков (Include Guards) —

конструкция препроцессора, защищающая от повторного включения одного и того же заголовочного файла. Предотвращают множественные определения функций и классов, которые вызывают ошибки компиляции.

Два способа реализации:

Через директиву `#pragma once`

Point.h

```
#pragma once //просто,
//но не стандартно
struct Point
{
  double x;
  double y;
};
```

Через конструкцию `#ifndef ... #endif`

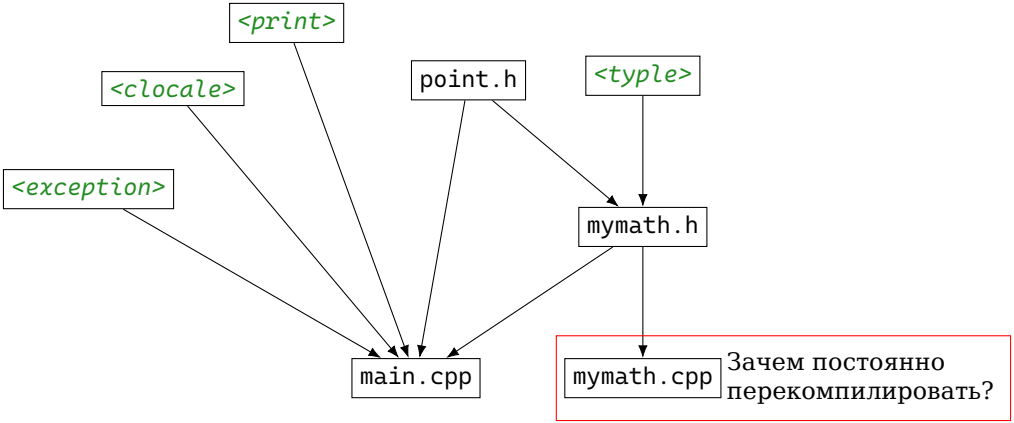
Point.h

```
#ifndef POINT_H //сложнее, но всегда работает
#define POINT_H //нужно придумать такое имя,
//которое никогда не повторится
struct Point
{
  double x;
  double y;
};

#endif //POINT_H
```

Статические библиотеки

Статическая библиотека (static library) —

это набор скомпилированных объектных файлов (`.o` или `.obj`), объединенных в один архивный файл (с расширением `.lib` в Windows или `.a` в Linux/macOS), который связывается с программой на этапе компоновки (линковки).



Команды Cmake для работы с библиотеками

Объявление библиотеки

```
add_library(<name> [<type>] [EXCLUDE_FROM_ALL] <sources> ... )
```

type:

STATIC — статическая библиотека
SHARED — динамическая библиотека
MODULE — модуль

Подключение библиотеки

```
target_link_libraries(<target> ... <item> ... ... )

target_link_libraries(<target>
  {INTERFACE|PUBLIC|PRIVATE} <item> ...
  [{INTERFACE|PUBLIC|PRIVATE} <item> ... ] ... )
```

target — к чему подключаем (программа или библиотека)
item — что подключаем (библиотека)

Создадим библиотеку mymath

CMakeLists.txt

```
cmake_minimum_required(VERSION 4.2)
project ("quadratic equation")
set (CMAKE_CXX_STANDARD 23)

#Создаем библиотеку
add_library(mymath mymath.cpp)
#Создаем саму программу
add_executable(qe main.cpp )
#Подключаем
target_link_libraries(qe mymath)
```

```
PS ... > cmake -B build ; cmake --build build/
 ...
mymath.vcxproj -> ... /build/Debug/mymath.lib   <== библиотека!
qe.vcxproj -> ... /build/Debug/qe.exe   <== программа!

PS ... >  ./build/Debug/qe.exe
/colororange1 -5 6
x1=3 x2=2x1 is not odd        <== все работает!
```

Создадим библиотеку mymath

CMakeLists.txt

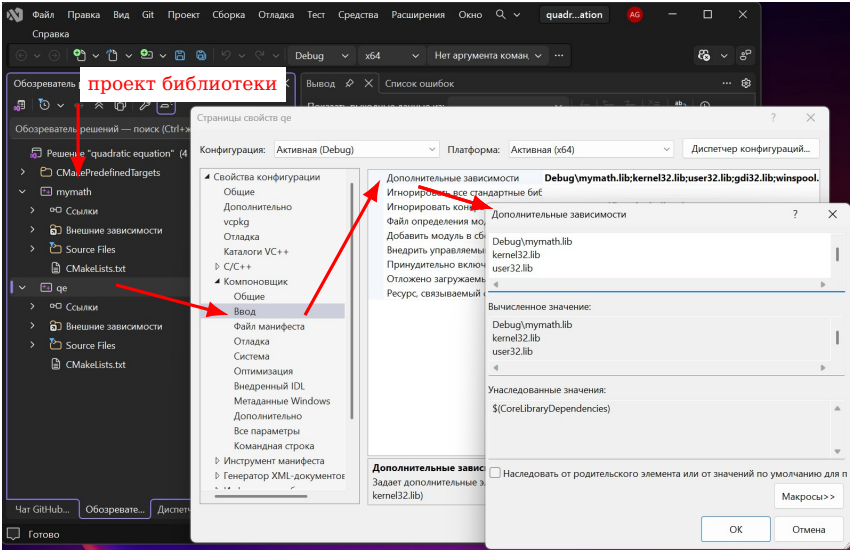
```
cmake_minimum_required(VERSION 4.2)
project ("quadratic equation")
set (CMAKE_CXX_STANDARD 23)

#Создаем библиотеку
add_library(mymath mymath.cpp)
#Создаем саму программу
add_executable(qe main.cpp )
#Подключаем
target_link_libraries(qe mymath)
```

```
PS ...> cmake -B build ; cmake --build build/
 ...
mymath.vcxproj -> .../build/Debug/mymath.lib  ⟸ библиотека!
qe.vcxproj -> .../build/Debug/qe.exe  ⟸ программа!

PS ...>  ./build/Debug/qe.exe
/colororange1 -5 6
x1=3 x2=2x1 is not odd       ⟸ все работает!
```

Проведём эксперимент

```
PS ...> mkdir new_folder

Directory: C:/...

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----          12.03.2026     1:18                new_folder

PS ...> cp ./build/Debug/qe.exe ./new_folder/
PS ...> dir ./new_folder/

Directory: C:/.../new_folder

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a---          12.03.2026     1:05        1106432 qe.exe
PS ...> ./new_folder/qe.exe
/colororange1 -5 6
x1=3 x2=2x1 is not odd        ⟸ все работает!
PS ...> /colororangerm -r ./new_folder
```

Как это выглядит в IDE



Динамические библиотеки

Динамическая библиотека (разделяемая библиотека, shared library) —

это файл, содержащий скомпилированный код и данные, который загружается в память во время выполнения программы, а не включается непосредственно в исполняемый файл.

Основные отличия от статических библиотек:

- Линковка на этапе выполнения.
- Машинный код расположен в отдельном файле.
- Загружаются в память по потребности.
- Одна библиотека может многократно использоваться разными программами.
- Работает медленнее.
- Могут быть проблемы с совместимостью.

Создадим динамическую библиотеку mymath

CMakeLists.txt

```
cmake_minimum_required(VERSION 4.2)
project ("quadratic equation")
set (CMAKE_CXX_STANDARD 23)

#Создаем динамическую библиотеку
add_library(mymath SHARED mymath.cpp) #добавили SHARED
#Создаем саму программу
add_executable(qe main.cpp )
#Подключаем
target_link_libraries(qe mymath)
```

Создадим динамическую библиотеку mymath

CMakeLists.txt

```
cmake_minimum_required(VERSION 4.2)
project ("quadratic equation")
set (CMAKE_CXX_STANDARD 23)

#Создаем динамическую библиотеку
add_library(mymath SHARED mymath.cpp) #добавили SHARED
#Создаем саму программу
add_executable(qe main.cpp )
#Подключаем
target_link_libraries(qe mymath)
```

Подправим код библиотеки для dll

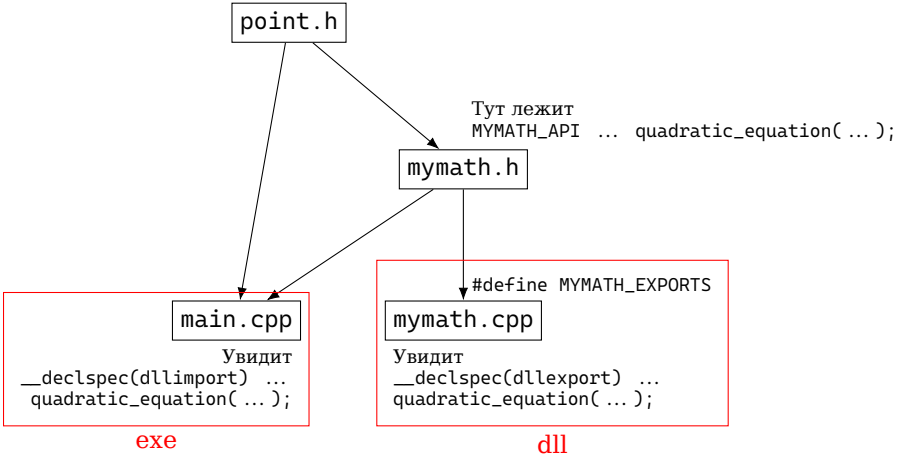
mymath.h

```
#include <tuple>
#include "point.h"

// Этот макрос будет экспортировать функции из DLL
#ifdef MYMATH_EXPORTS
  #define MYMATH_API __declspec(dllexport) //функция экспортируется
#else
  #define MYMATH_API __declspec(dllimport) //функция импортируется
#endif

MYMATH_API std::tuple<double,double> quadratic_equation(double, double, double);

MYMATH_API inline bool isEven(int n) {
  return n % 2 == 0;
}
```

mymath.cpp

```
...
#define MYMATH_EXPORTS //Добавляем макрос
#include "mymath.h"
...
```

Схема проекта с динамической библиотекой



MYMATH_API ⇒ __declspec(dllimport) в exe

MYMATH_API ⇒ __declspec(dllexport) в dll

Результат

```
PS ... > rm -r ./build/ -Force; cmake --build build/; cmake --build build/
...
Создается библиотека ... /build/Debug/mymath.lib и объект lib!! o_O
... /build/Debug/mymath.exp mymath.vcxproj -> ... /build/Debug/mymath.dll
...
qe.vcxproj -> ... /build/Debug/qe.exe
PS ... > ./build/Debug/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
PS ... > mkdir new_folder
PS ... > cp ./build/Debug/qe.exe ./new_folder/
PS ... > ./new_folder/qe.exe
PS ... >                    <== ничего не работает =(
PS ... > cp ./build/Debug/mymath.dll ./new_folder/
PS ... > ./new_folder/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
```

Результат

```
PS ... > rm -r ./build/ -Force; cmake --build build/; cmake --build build/
 ...
Создается библиотека ... /build/Debug/mymath.lib и объект  lib!! o_O
 ... /build/Debug/mymath.exp mymath.vcxproj -> ... /build/Debug/mymath.dll
 ...
qe.vcxproj -> ... /build/Debug/qe.exe
PS ... > ./build/Debug/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
PS ... > mkdir new_folder
PS ... > cp ./build/Debug/qe.exe ./new_folder/
PS ... > ./new_folder/qe.exe
PS ... >                          ⟸ ничего не работает =(
PS ... > cp ./build/Debug/mymath.dll ./new_folder/
PS ... > ./new_folder/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
```

Результат

```
PS ... > rm -r ./build/ -Force; cmake --build build/; cmake --build build/
...
Создается библиотека ... /build/Debug/mymath.lib и объект  lib!! o_O
... /build/Debug/mymath.exp mymath.vcxproj -> ... /build/Debug/mymath.dll
...
qe.vcxproj -> ... /build/Debug/qe.exe
PS ... > ./build/Debug/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
PS ... > mkdir new_folder
PS ... > cp ./build/Debug/qe.exe ./new_folder/
PS ... > ./new_folder/qe.exe
PS ... >                          ⟸ ничего не работает =(
PS ... > cp ./build/Debug/mymath.dll ./new_folder/
PS ... > ./new_folder/qe.exe
1 -5 6
x1=3 x2=2x1 is not odd
```